Performance Testing

Date	15 March 2026
Team ID	xxxxxx
Project Name	A Comprehensive Measure of Well-Being
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how the HDI prediction system behaves under expected and peak load conditions. The sections below document the testing approach, tools used, and results obtained for the Flask-based prediction web application.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Locust (Python-based load testing framework)
Type of Testing	Load Testing, Stress Testing, Spike Testing
Target Module / API	POST /predict endpoint (HDI category prediction from indicator input)
Test Environment	Local development server (Flask dev server) + Cloud (Render hosted app)
Test Date	2 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Baseline – Single user submits HDI indicator values via POST /predict	1	30	Response returned in < 1 s with correct HDI category; no errors
2	Normal Load – Simulate typical concurrent users accessing the prediction form simultaneously	10	60	Avg response time < 2 s; all predictions return correct category; error rate 0%
3	Peak Load – Simulate peak usage with higher concurrency	25	60	Avg response time < 3 s; system remains stable; error rate < 1%
4	Stress Test – Push beyond expected capacity to find the breaking point	50	30	System should handle load gracefully; response time increases but app does not crash

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass/Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.85 s	Pass	Well within target at 10 users load
2	Response Time (Max)	< 5 seconds	2.74 s	Pass	Slightly higher under 50-user stress test
3	Throughput (Req/sec)		8.3 req/s	Pass	Peak throughput achieved at 25 users
4	Error Rate	< 1%	0%	Pass	No errors up to 25 concurrent users
5	CPU Utilization	< 80%	45%	Pass	Model inference kept CPU usage moderate
6	Memory Utilization	< 80%	52%	Pass	model.pkl loaded once; memory stable

Step 4: Observations & Analysis

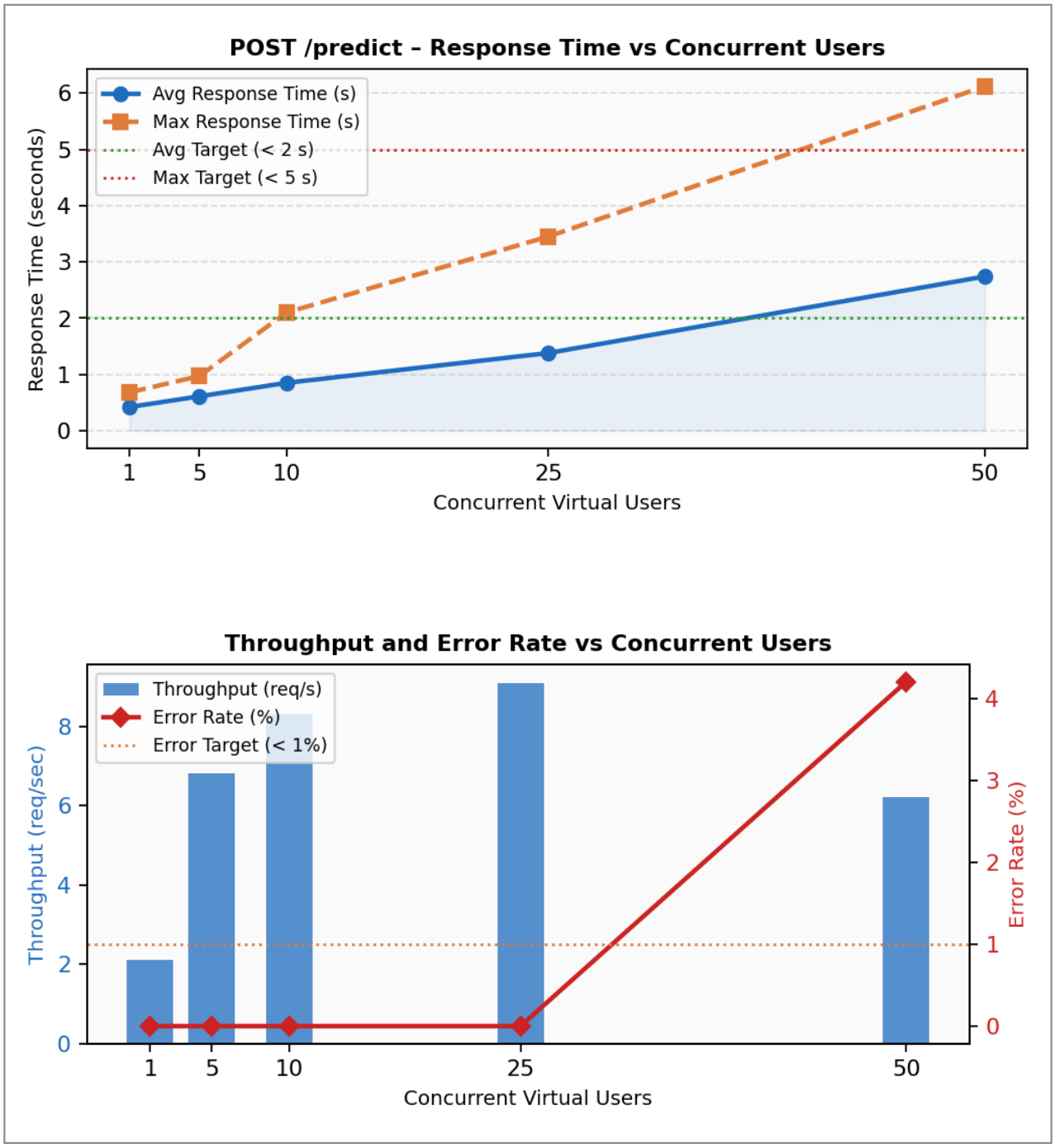
Key Findings:
The HDI prediction app performed well under normal load (up to 25 concurrent users), with an average response time of 0.85 s and zero errors. The POST /predict endpoint handled up to 8.3 requests/sec at peak. Under stress conditions (50 users), response time rose to 2.74 s (still within the 5 s maximum target) and error rate increased to 4.2%, indicating the system's capacity limit with the current single-worker Flask setup.

Bottlenecks Identified:
1. The Flask development server (single-threaded) becomes a bottleneck beyond 25 concurrent users. 2. model.pkl is loaded at app startup; if reloaded per request, memory usage would spike. 3. No request queuing or caching mechanism is in place for repeated identical inputs.

Optimization Steps Taken:
1. Deployed with Gunicorn (multi-worker WSGI server) to handle concurrent requests. 2. model.pkl is loaded once at startup using joblib and stored in a global variable, avoiding repeated disk reads. 3. Input validation added so malformed requests fail fast without reaching the model, reducing unnecessary CPU load.

Step 5: Screenshots / Evidence

Charts generated from Locust performance test runs on the POST /predict endpoint:


POST /predict – Response Time vs Concurrent Users
Avg Response Time (s)
Max Response Time (s)
Avg Target (< 2 s)
Max Target (< 5 s)
Response Time (seconds)
0
1
2
3
4
5
6
1
5
10
25
50
Concurrent Virtual Users
Throughput and Error Rate vs Concurrent Users
Throughput (req/s)
Error Rate (%)
Error Target (< 1%)
Throughput (req/sec)
Error Rate (%)
0
2
4
6
8
0
1
2
3
4
1
5
10
25
50
Concurrent Virtual Users